

T.POOJITHA (192412318)

ITA0609

Exp4 Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

import numpy as np

Input Dataset (XOR)

X = np.array([[0,0],

[0,1],

[1,0],

[1,1]])

Output Dataset

y = np.array([[0],

[1],

[1],

[0]])

Sigmoid Activation Function

def sigmoid(x):

return 1/(1+np.exp(-x))

Derivative of Sigmoid

def sigmoid_derivative(x):

return x*(1-x)

Initialize weights and biases

np.random.seed(1)

input_neurons = 2

hidden_neurons = 2

output_neurons = 1

wh = np.random.uniform(size=(input_neurons, hidden_neurons))

bh = np.random.uniform(size=(1, hidden_neurons))

wo = np.random.uniform(size=(hidden_neurons, output_neurons))

bo = np.random.uniform(size=(1, output_neurons))

learning_rate = 0.5

output

Copy to Drive

```
output = sigmoid(np.dot(hidden, wo) + bo)

print("\nTest Input:", test)
print("Predicted Output:", np.round(output,3))
```

```
... Predicted Output:
[[0.019]
 [0.983]
 [0.983]
 [0.017]]
```

```
Test Input: [[1 0]]
Predicted Output: [[0.983]]
```

```

# Training using Backpropagation

for epoch in range(10000):

    # Forward Propagation

    hidden_input = np.dot(X, wh) + bh

    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, wo) + bo

    predicted_output = sigmoid(final_input)

    # Error

    error = y - predicted_output

    # Backpropagation

    d_output = error * sigmoid_derivative(predicted_output)

    error_hidden = d_output.dot(wo.T)

    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    # Update Weights

    wo += hidden_output.T.dot(d_output) * learning_rate

    bo += np.sum(d_output, axis=0, keepdims=True) * learning_rate

    wh += X.T.dot(d_hidden) * learning_rate

    bh += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

# Results

print("Predicted Output:")

print(np.round(predicted_output,3))

# Test New Sample

test = np.array([[1,0]])

hidden = sigmoid(np.dot(test, wh) + bh)

output = sigmoid(np.dot(hidden, wo) + bo)

print("\nTest Input:", test)

print("Predicted Output:", np.round(output,3))

```